Introduction to Joins:

- A join is a query that combines rows from two or more tables based on a condition based on common column
- Join achieves the goal of creating a single SQL query that can fetch data from two or more tables.
- The SELECT statement can select any of the columns from any of the tables referred in from clause in Join query

Different types of SQL joins :

There are different types of joins:

- CROSS JOIN
- INNER JOIN
- EQUI JOIN
- NON EQUI JOIN
- OUTER JOIN
  - LEFT OUTER JOIN
  - RIGHT OUTER JOIN
  - FULL OUTER JOIN
- SELF JOIN

Guidelines:

When writing a SELECT statement that joins tables, if the same column name appears in more than one table, the column name must be prefixed with the table name/Alias name of the table. To join 'N' tables , a minimum of 'N-1' join conditions are required. Consider the below tables:

Consider the below three Tables to understand the joins working mechanism:

1.Cross Join:

The Cross join / Cartesian product is a join query , that does not contain a join condition. Oracle combines each row of one table with each and every row of the other.

Syntax:

SELECT * FROM tableA , tableB;

Ex:

*SELECT * FROM Employee, Department;*

The above query returns 35 records. Each row of of Employee table is linked with each and every row of Department table . So the 101 (Emp Id) record from Employee table appears 5 times ie once for each record of the Department table. So 101 is appearing in 5 rows in the resultant record set, as below – Some records from result set

2.Inner Join:

An inner is used when the join fields are guaranteed not to be NULL.

2.a) Equi Join : Comes under Inner join category

It is a join condition containing an equality operator (=) .

This join condition combines/fetch the data from the matched records of two tables (Tow copies of same table in case of self-join) based on the common columns from the two tables ,on which the join condition is applied .

Syntax :

SELECT tableA.col1, tableA.col2, tableB.col1,..... FROM
tableA, tableB WHERE tableA.col1 = tableB.col1;

Ex: To get Department name for each Employee along with the Employee details

Select EMP_ID,EMP_NAME,EMP_SAL,EMP_LOC,EMP_MGR_ID,EMP_JOB, EMP_DEPT_ID,DEPT_NAME from Employee,Department where Employee.EMP_DEPT_ID=Department.DEPT_ID;

The above query return multiple records where each record contains the data from the matched records of Employee and Department tables , based on the join condition formed on common column between the two tables

i.e .. For each record of employee table, if there is a matched record in department table with reference to the department Id , then the Query will fetch some data from Employee and some data from department table

Output for the above Query:

In the above output *EMP_ID,EMP_NAME,EMP_SAL,EMP_LOC, EMP_MGR_ID, EMP_JOB, EMP_DEPT_ID are fetched from employee table and DEPT_NAME* from Department table i.e., the rows from both the tables where **Department Id** is matching.

Note:

Table Name is not mandatory to prefix before the Column in the Select Statement column list , if the column name is not same across two tables , from which the data is being fetched.

2.b) Non Equi Join:

It is a join condition that is executed when no column in one table is directly linked with a column in another table. The data in the tables are logically related through appropriate values.

Syntax :

SELECT tableA.col1, tableA.col2, tableB.col1,..... FROM
tableA, tableB WHERE tableA.col1 BETWEEN tableB.col2 AND tableB.col3;

Example:

*select * from Employee ,Grade where Employee.Emp_sal between grade.LowSal and grade.HighSal;*

The above query retrieves the employee details along with his salary grade based on the join condition.

3.Outer Join:

Outer join extends the result of Equi join. Outer join returns all the rows from both the tables that satisfy the join condition and also the rows from one table which do not satisfy the join condition. The Outer join operator is (+) , which is used on one side of the join condition.

3.a) Left Outer Join:

It is a join which returns all the rows from the left hand side of the table specified at the join condition and only those rows from the other table which are matching with the join condition.

Syntax : SELECT tableA.col1,tableA.col2,tableB.col1,..... FROM tableA,tableB WHERE tableA.col1 = tableB.col1(+);

Example:

Retrieve all the employee details along with their department details.

If employees are already tagged then get the tagged department details along with the employee details , If not tagged to any department then get atleast the employee details

Query:

```
select EMP_ID,EMP_NAME,EMP_SAL,EMP_LOC,EMP_MGR_ID,EMP_JOB,
EMP_DEPT_ID,DEPT_Name from Employee,Department where
Employee.EMP_DEPT_ID=Department.DEPT_ID(+);
```

The above query displays all the records which are matching with the join condition from both the tables along with the rows from employee table which are not matched with department table.

Output from the above Query:

If we notice the above output, for the first 5 records, there is a match of **Department Id** between two tables. Therefore, the Query retrieves the data from the matched records from both the tables. However, in case of last two records from Employee table, there are no records in the department table having

the same department id, so there is no match found in department table. Hence, only the data from Employee table is displayed.

3.b) Right Outer Join:

It is a join which returns all the rows from the right hand side of the table specified at the join condition and only those rows from the other table which are matching with the join condition.
Syntax : SELECT tableA.col1,tableA.col2,tableB.col1,..... FROM tableA,tableB WHERE tableA.col1(+) = tableB.col1;

Example: Fetch the department details for all the departments with the details of Employee, for all employees tagged to a department. If for any department, there is no employee, fetch the department details only.

Query:

select EMP_ID,EMP_NAME,EMP_SAL,EMP_LOC,EMP_MGR_ID,EMP_JOB, EMP_DEPT_ID, DEPT_Name from Employee,Department where Employee.EMP_DEPT_ID(+)=Department.DEPT_ID;

Output from the above Query:

If we notice the above output, for the first 5 records, there is a match of **Department Id** between two tables. Hence, the Query retrieves the data from the matched records from both the tables. However, for the last two records of Department table, there are no record in the Employee table having that department Id. Hence, only the data from Department table is displayed .

3.c) Full Outer Join:

It is a join which returns all the rows from both the tables which are placed on either side of the join condition.

Syntax : SELECT tableA.col1,tableA.col2,tableB.col1,..... FROM tableA,tableB WHERE tableA.col1 = tableB.col1(+)
UNION
Syntax : SELECT tableA.col1,tableA.col2,tableB.col1,..... FROM tableA,tableB WHERE tableA.col1(+) = tableB.col1;

Example:
Retrieve all the records from Employee and Department table in such a way that, if there is a match between the tables with reference to Department Id, then get the resulted record from two tables else fetch the employee data though they have not tagged and Department data though there is no single employee tagged for the department

Query for the above requirement:

The above query displays all the records, which fulfill the join
condition along with the rows from both employee and department tables which do not fulfill the join condition.

Output:

4.Self Join:

It is a join used to join a table to itself. Single table is logically considered as two tables and joined .

Syntax : SELECT A.* FROM tableA A,tableA B WHERE A.col1 = B.col1 AND condition;

Example:

GET the names of the Managers for each Employee from Employee table.

In the Employee table, Manager name is not available as a column in the table. However, every Manager is an Employee. For each row , the value for the attribute Emp_Mgr_Id is the value of the emp_Id of attribute the employee who is the manager for that employee.

To solve the query, we have to join the Employee table to itself and compare the Manager Id of each Employee with the Employee IDs of the other copy of the table and get the EmpName accordingly from the second copy of the table which is the manager Name

Query for the above Requirement:

SELECT A.EMP_ID,A.EMP_NAME,A.EMP_SAL,A.EMP_LOC,A.EMP_MGR_ID,B.Emp_Name Manager_Name from Employee A,Employee B where A.EMP_MGR_ID=B.EMP_ID;

Output:

The above query is going to display the details of Manager for each Employee